PLATFORM REFERENCE ARCHITECTURE · GITOPS BOOTSTRAP

Five resource planes, one gated rollout.

A design for organising the **platform-infra** Argo CD Applications as a plane-based reference architecture — so every component has an owner, and the platform boots itself in dependency order.

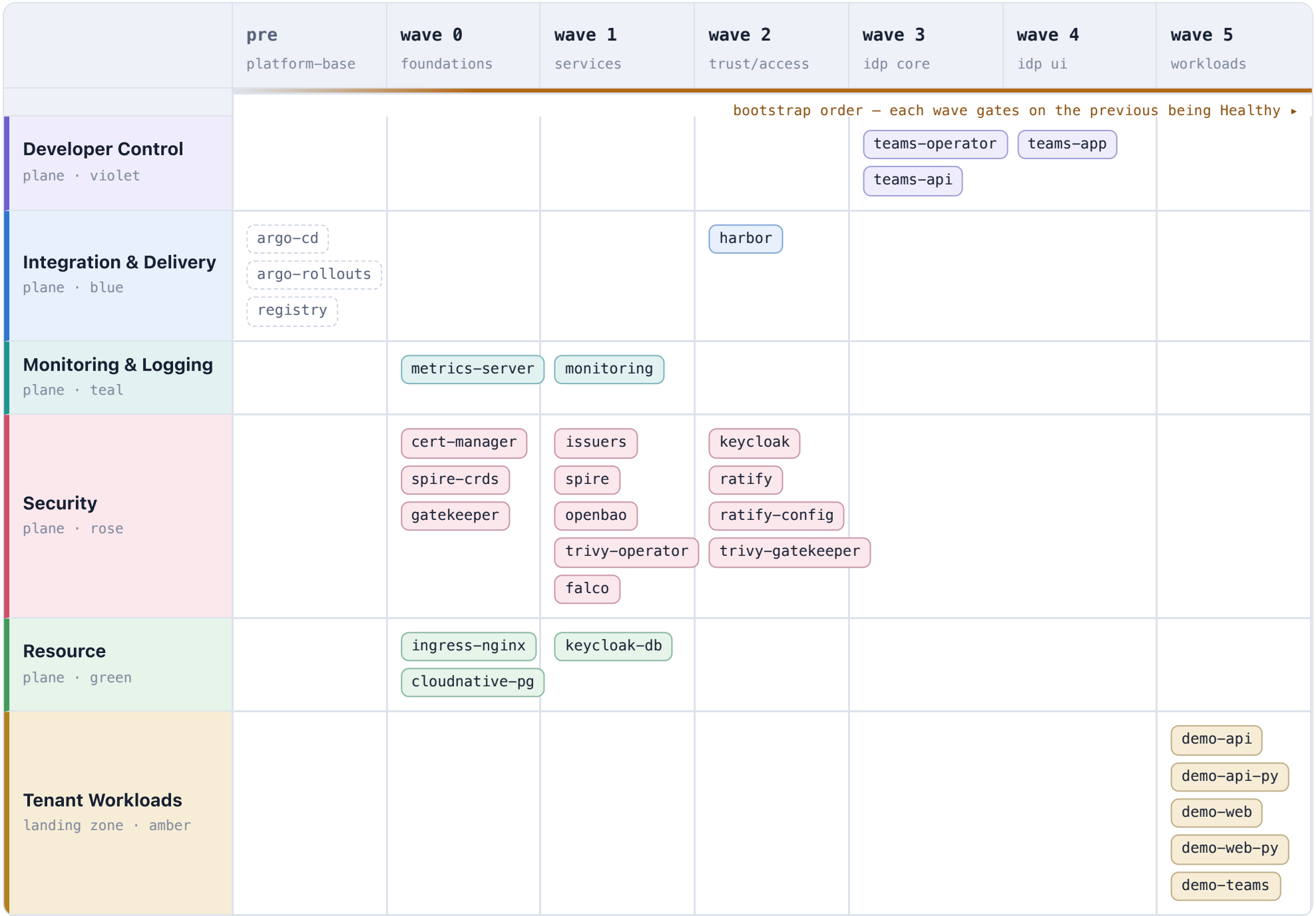
▤ group by **plane** · ▸ order by **wave** — two orthogonal axes, kept separate.

26 Applications | 5 planes + tenant landing zone | app-of-apps · sync-waves · AppProjects | repo: rkdutta/platform-infra

THE MAP

Plane × wave matrix

Rows are **planes** — the ownership/grouping axis. Columns are **bootstrap waves** — the ordering axis. Every app lives in exactly one cell. Read across a row to see what a plane owns; read down a column to see everything that installs together. Notice wave 0 spans three planes at once — which is exactly why grouping and ordering can’t be the same axis.


pre
platform-base
wave 0
foundations
wave 1
services
wave 2
trust/access
wave 3
idp core
wave 4
idp ui
wave 5
workloads
bootstrap order — each wave gates on the previous being Healthy ▸
Developer Control
plane · violet
teams-operator
teams-app
teams-api
Integration & Delivery
plane · blue
argo-cd
argo-rollouts
registry
harbor
Monitoring & Logging
plane · teal
metrics-server
monitoring
Security
plane · rose
cert-manager
spire-crds
gatekeeper
issuers
spire
openbao
trivy-operator
falco
keycloak
ratify
ratify-config
trivy-gatekeeper
Resource
plane · green
ingress-nginx
cloudnative-pg
keycloak-db
Tenant Workloads
landing zone · amber
demo-api
demo-api-py
demo-web
demo-web-py
demo-teams

Developer Control · Integration & Delivery · Monitoring & Logging · Security · Resource · Tenant Workloads

THE CORE DECISION

Group and order are different questions

Every earlier attempt stalled because a single number was asked to answer both “who owns this?” and “when does it install?”. It can’t. Split them, and each becomes simple.

▤ GROUPING AXIS

Planes — stable, org-shaped

Which plane a component belongs to answers ownership, RBAC, and cognitive load. It rarely changes. cert-manager is *always* Security, wherever it sits in the boot sequence.

`expressed by → directory · AppProject · plane label`

▸ ORDERING AXIS

Waves — dependency layers

When a component installs is a dependency question, and it cuts *across* planes: ingress (Resource) and cert-manager (Security) must both be up before anything else — same wave, different planes.

`expressed by → argocd.argoproj.io/sync-wave`

THE GROUPING AXIS

The five planes + a tenant landing zone

Based on the CNCF / Humanitec platform reference architecture. Each plane becomes a directory, an app-of-apps parent, and an AppProject that pins its apps to the platform-infra repo and allowed destinations.

DEVELOPER CONTROL

The interfaces developers touch

Self-service portal, golden paths, the workload API that turns a team's intent into deployments.

`teams-operator` `teams-api` `teams-app`

project developer-control **apps** 3

INTEGRATION & DELIVERY

Intent → running software

The GitOps engine, progressive-delivery controller and artifact registry. The engine itself ships from platform-base; Harbor is the in-cluster registry.

`argo-cd · base` `argo-rollouts · base` `registry · base` `harbor`

project integration-delivery **apps** 1 (+3 base)

MONITORING & LOGGING

Signals about health

Cluster metrics for autoscaling and progressive-delivery analysis, plus the Prometheus/Grafana observability stack.

`metrics-server` `monitoring`

project monitoring **apps** 2

SECURITY

Identity · policy · secrets · PKI · supply-chain trust · runtime defense

The broadest plane: certificate authority and workload identity, human SSO, secrets, admission policy, image-signature verification, vulnerability scanning and runtime threat detection.

`cert-manager` `issuers` `spire-crds` `spire` `keycloak` `openbao` `gatekeeper` `ratify` `ratify-config` `trivy-operator` `trivy-gatekeeper` `falco`

project security — **apps** 12

RESOURCE

The provisioned substrate

Networking edge and data-provider operators — the resources platform and tenant services bind to. Keycloak's database lives here, not with Keycloak.

`ingress-nginx` `cloudnative-pg` `keycloak-db`

project resource — **apps** 3

TENANT WORKLOADS · LANDING ZONE

Deployed onto the platform, not part of it

The demo apps are consumers delivered *through* the Developer Control Plane. Kept separate so platform and tenant lifecycles never entangle.

`demo-api` `demo-api-py` `demo-web` `demo-web-py` `demo-teams`

project tenant-workloads — **apps** 5

THE ORDERING AXIS

Bootstrap sequence — six gated waves

The root app-of-apps applies each wave, then **waits until every Application in it is Synced and Healthy** before starting the next. Because an Argo `Application` is itself a health-checked resource, "Healthy" means the underlying workloads are genuinely up — this is your "first set running before the rest start", enforced by Argo CD, no extra tooling.

0	FOUNDATIONS	**Cluster primitives — routing, PKI, policy, data operator, metrics** Gate: CRDs established · admission webhooks Ready · ingress routable · issuer issuing. `ingress-nginx` `cloudnative-pg` `cert-manager` `spire-crds` `gatekeeper` `metrics-server`
1	SERVICES	**Platform services that depend on the foundations** Gate: secrets backend unsealed · identities issued · scanners reporting. `issuers` `spire` `openbao` `trivy-operator` `falco` `keycloak-db` `monitoring`
2	TRUST / ACCESS	**Identity, supply-chain trust and the registry** Gate: SSO reachable · signature verification enforcing · registry serving. `keycloak` `ratify` `ratify-config` `trivy-gatekeeper` `harbor`
3	IDP CORE	**Developer Control Plane — control loop & API** Gate: operator reconciling Team CRs · API authenticating against Keycloak. `teams-operator` `teams-api`

4 IDP UI	**Developer Control Plane — the portal** Gate: portal served through ingress, talking to teams-api. `teams-app`
5 WORKLOADS	**Tenant landing zone — demo applications** Gate: platform fully green; workloads deploy under policy & identity. `demo-api` `demo-api-py` `demo-web` `demo-web-py` `demo-teams`

JUDGEMENT CALLS

Cross-cutting rulings

Four components sit on a boundary. These are deliberate assignments, made so ownership stays unambiguous.

`falco — Observability or Security?`

Security. A detective control that happens to emit signals; owned by the security team, not the SRE dashboard.

`keycloak vs keycloak-db`

Keycloak is IAM → **Security**. Its Postgres is a resource → Resource plane. The cross-plane edge is real; don't hide it.

`ratify — Delivery or Security?`

Security. Supply-chain *trust* enforced at admission. It gates delivery but the policy is owned by security.

`demo-* — a plane?`

No. They're tenant workloads deployed onto the platform. A landing zone, outside the five platform planes.

HOW IT LANDS IN GIT

Repository structure & rollout mechanism

Planes become directories and parent Applications; the root wires them together; AppProjects draw the guardrails. Argo CD reconciles the rest.

```
# hierarchical app-of-apps — root → plane parents → leaf apps
platform-infra/
  bootstrap/
    root-app.yaml            # owns the 6 plane parents; ordered entrypoint
    repo-credentials.yaml    # private-repo Secret (token injected, not committed)
  projects/                  # one AppProject per plane → RBAC + source/dest guardrails
    security.yaml  resource.yaml  developer-control.yaml  …
  planes/                    # each = an app-of-apps over its apps/<plane>/ dir
    security.yaml  resource.yaml  monitoring.yaml  …
  apps/
    security/       # cert-manager/ spire/ keycloak/ gatekeeper/ ratify/ …
    resource/       # ingress-nginx/ cloudnative-pg/ keycloak-db/
    monitoring/     # metrics-server/ monitoring/
    integration-delivery/   # harbor/
    developer-control/      # teams-operator/ teams-api/ teams-app/
    workloads/      # demo-api/ demo-web/ demo-teams/ …

# leaf app carries both axes:
metadata:
  labels:    { plane: security }             # grouping
  annotations: { argocd.argoproj.io/sync-wave: "0" }  # ordering
spec:
  project: security                         # guardrail
```

GATING

Sync-waves run inside the root's sync; Argo CD holds each wave until its Applications report Healthy. Ordering is cross-plane by design.

PRIVATE REPO

A labelled `repository` Secret gives Argo CD read access. Apply it **before** the root app, or the first sync errors until creds land.

ONE APPLY

`kubectl apply -f bootstrap/root-app.yaml` once. After that it's pure GitOps — edit `apps/…`, push, reconcile.

DECISION RECORD

ADR-001 · Plane-based platform bootstrap

ADR-001 PROPOSED `supersedes: flat app-of-apps, ad-hoc waves`

CONTEXT	26 Argo CD Applications in platform-infra need an ordered, self-documenting bootstrap. Attempts to encode both ownership and rollout order in one dimension (folder, wave, or business-domain tier) kept colliding — foundational components share dependency waves but not domains.
DECISION	Adopt the CNCF/Humanitec five-plane reference architecture. **Group by plane** via directories + one AppProject per plane + a `plane` label. **Order by dependency layer** via `sync-wave` (0–5), allowed to cross planes. Model tenant workloads as a separate landing zone, deployed last.
CONSEQUENCES	+ Unambiguous ownership & RBAC boundary per plane (AppProject). + Deterministic, health-gated boot — "first set running before the rest", enforced by Argo CD. + Structure doubles as documentation; new components have an obvious home. − One-time repository restructure (26 apps move into plane dirs). − Cross-plane waves need discipline — the wave number lives with the app, reviewed on change. − A stuck plane blocks later waves — intended, but demands healthy components before promotion.

platform-infra · reference architecture v1 (proposed) · group by plane ▤ · order by wave ▸ · next: choose full restructure or overlay